

Memory Efficient Doubly Linked List

–Delete At End – Program Analysis

And Time Complexity.

Program:

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Traverse to end
    while (true)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr)
            break; // curr is the last node
        prev = curr;
        curr = next;
    }

    // curr is the last node. prev is the second to last.
    if (prev != nullptr)
    {
        // prev->npx was: XOR(prevPrev, curr)
        // Needs to be: XOR(prevPrev, NULL)

        // Decode prevPrev
        Node *prevPrev = XOR(prev->npx, curr);
```

```

        prev->npx = XOR(prevPrev, nullptr);
    }
    else
    {
        // List had only one node
        head = nullptr;
    }

    cout << "Deleted " << curr->data << " from the end." <<
endl;
    free(curr);
}

...

case 6:
    deleteFromEnd();
    break;

```

Program Analysis

1. When List Is Empty

```

if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}

```

Node * head

Address

Head

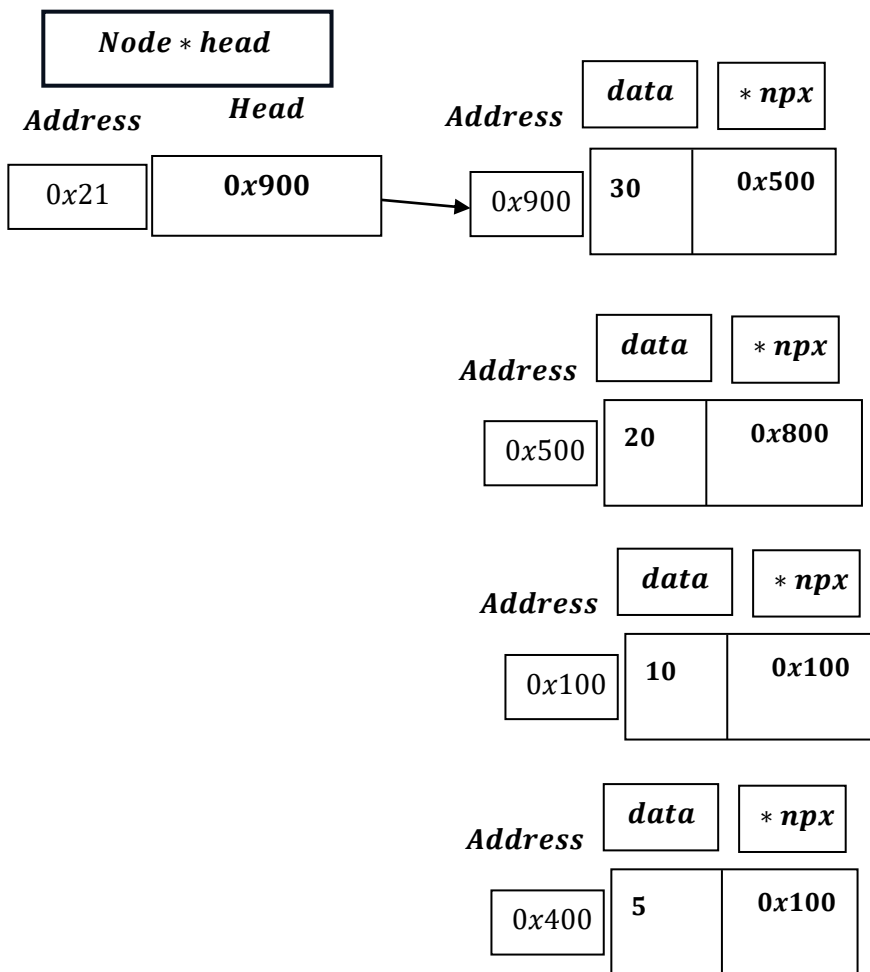
0x21

nullptr

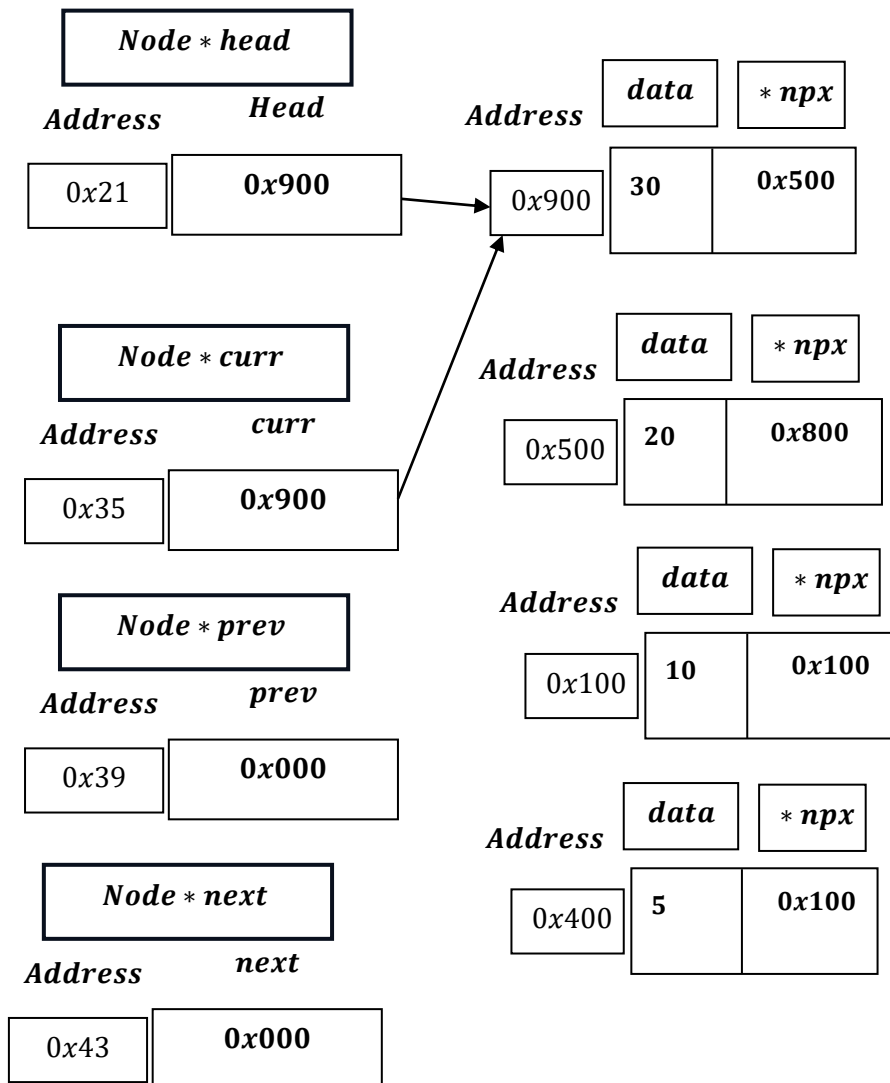
Then , it prints : "List is empty.Nothing to delete."

Return Void and Exit.

2. When List Is Not Empty



```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```



```
while (true)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr)
        break; // curr is the last node
    prev = curr;
    curr = next;
}
```

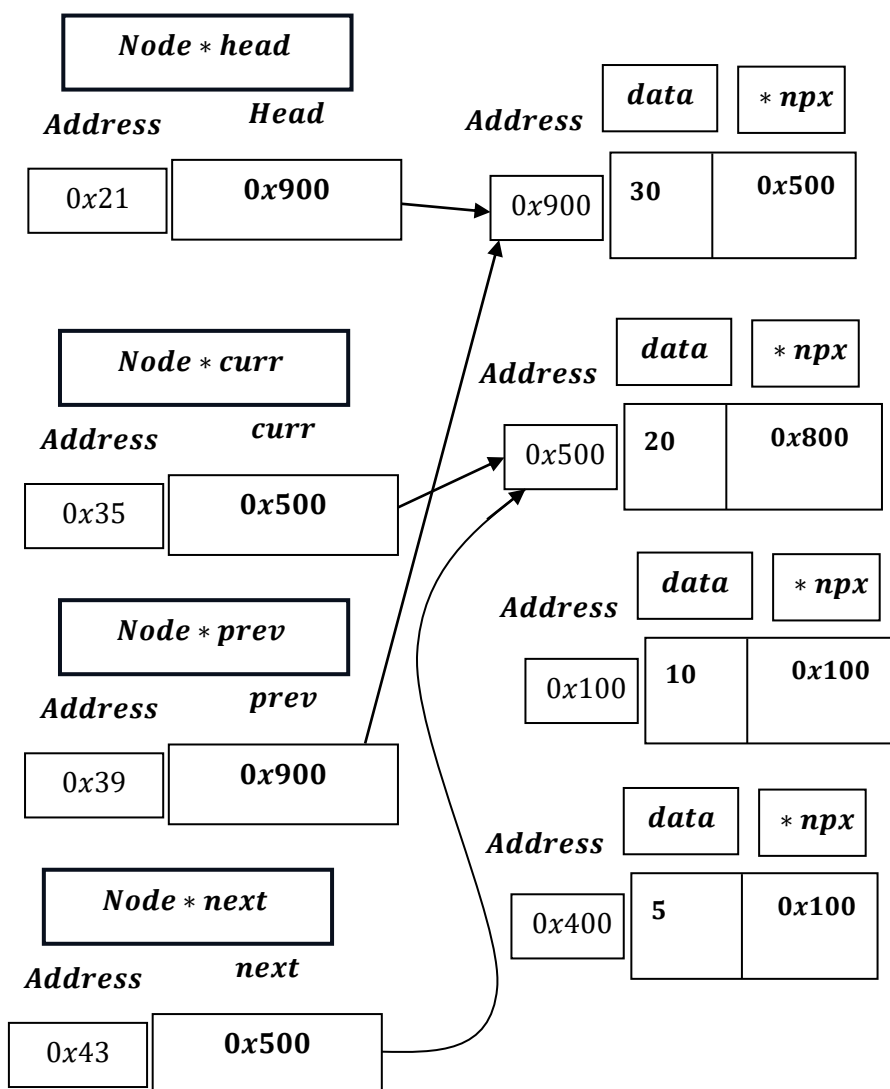
While(true):

next = XOR(prev: 0x000, curr → npx: 0x500) = 0x500.

if(next: 0x500 = nullptr: 0x000) is false.

prev = curr = 0x900

curr = next = 0x500.



While(true):

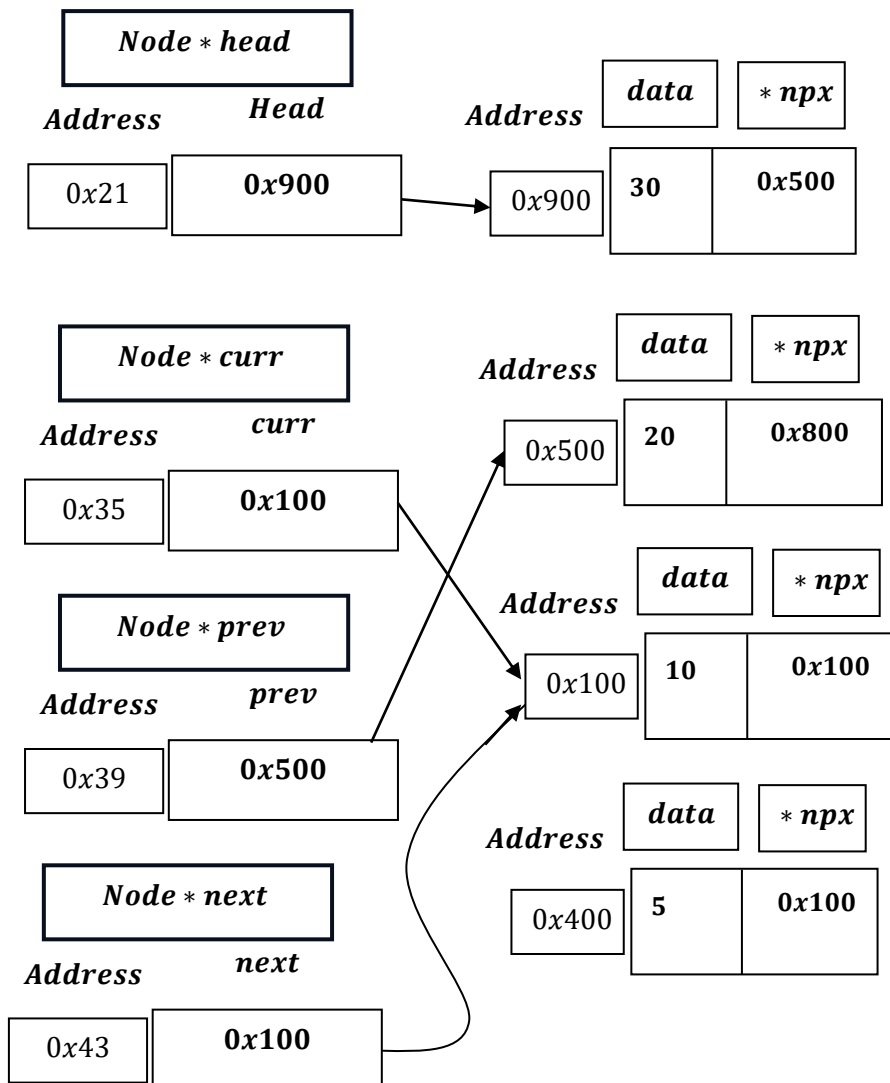
next = XOR(prev: 0x900, curr → npx: 0x800)

= 1001₂ ⊕ 1000₂ = 0001₂ = 0x100

if(next: 0x100 = nullptr: 0x000) is false.

prev = curr = 0x500

curr = next = 0x100.



While(true):

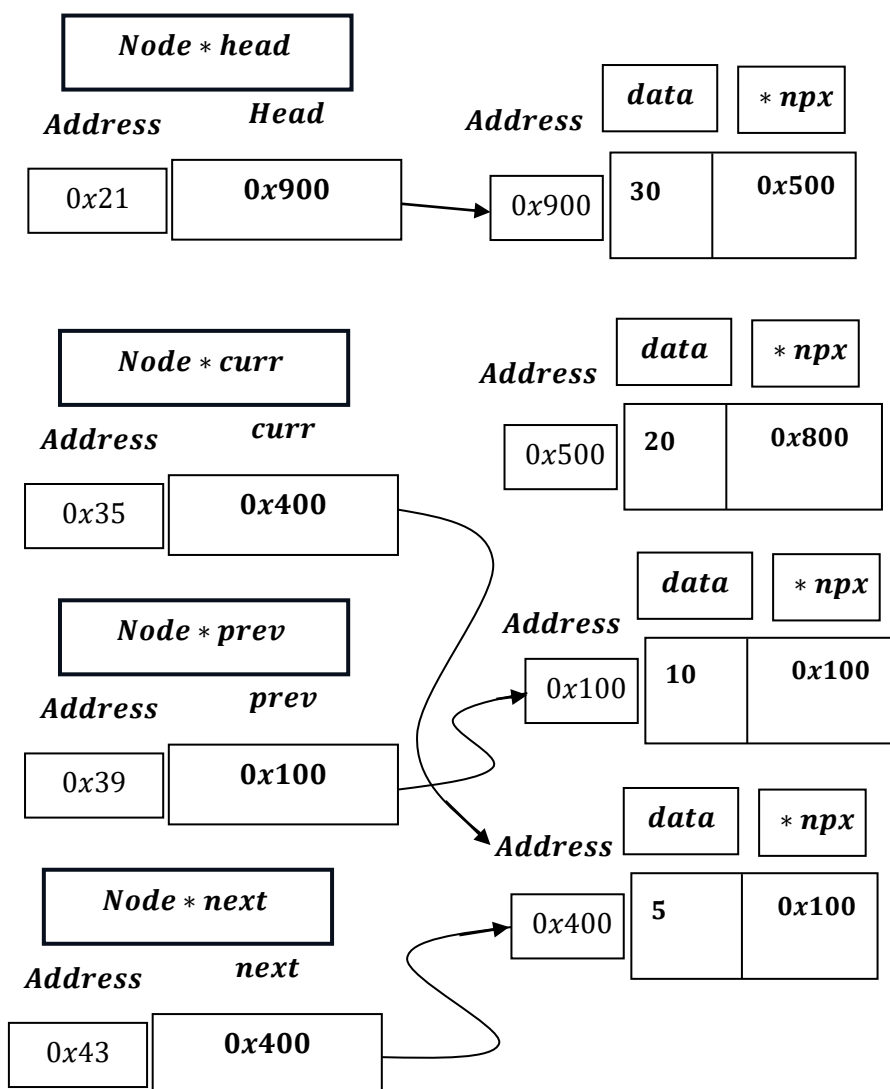
next = XOR(prev: 0x500, curr → npx: 0x100)

= 0101₂ ⊕ 0001₂ = 0100₂ = 0x400

if(next: 0x400 = nullptr: 0x000) is false.

prev = curr = 0x100

curr = next = 0x400.



While(true):

next = XOR(prev: 0x100, curr → npx: 0x100)

= 0001₂ ⊕ 0001₂ = 0000₂ = 0x000

if(next: 0x000 = nullptr: 0x000) is True:

break; → statement helps to exit from the loop.

... ..

... ..

```
if (prev != nullptr)
{
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, nullptr);
}
else{..... }
```

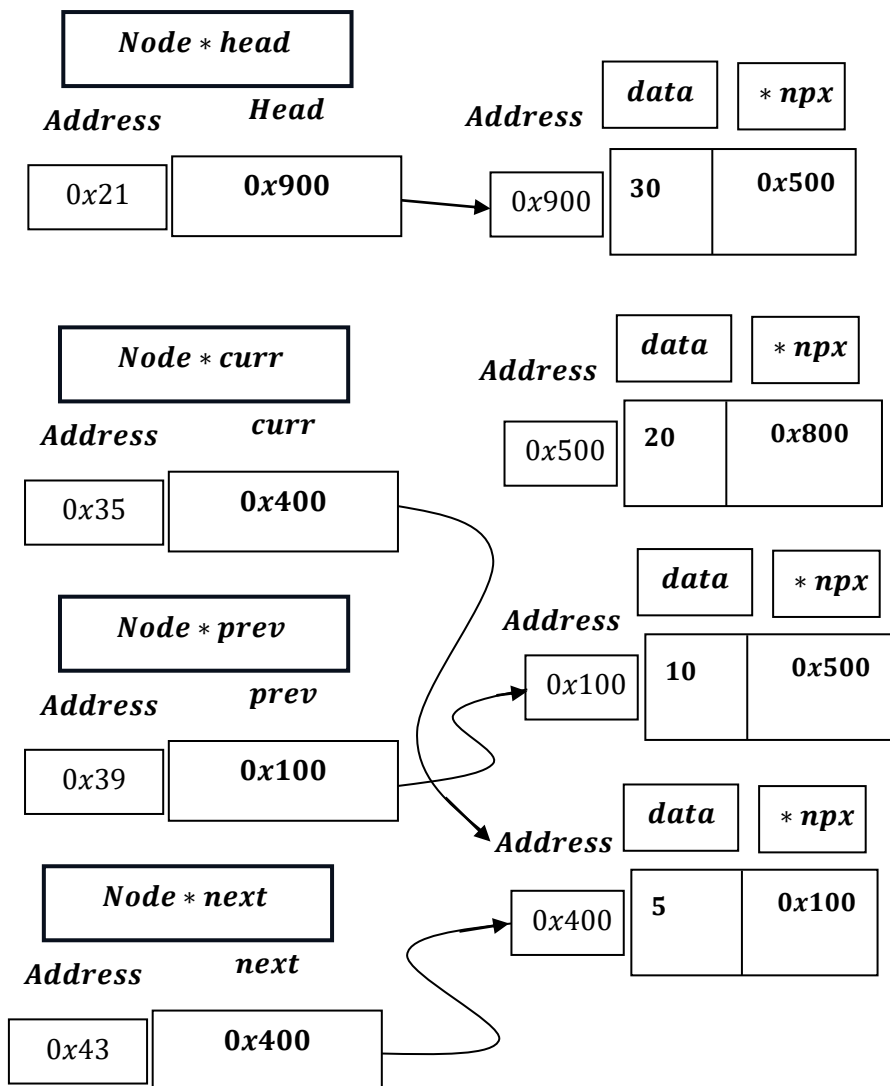
Prev: 0x100 ≠ NULLPTR[TRUE]:

Node * prevPrev = prev → npx: 0x100 ⊕ curr: 0x400

= 0001₂ ⊕ 0100₂ = 0101₂ = 0x500

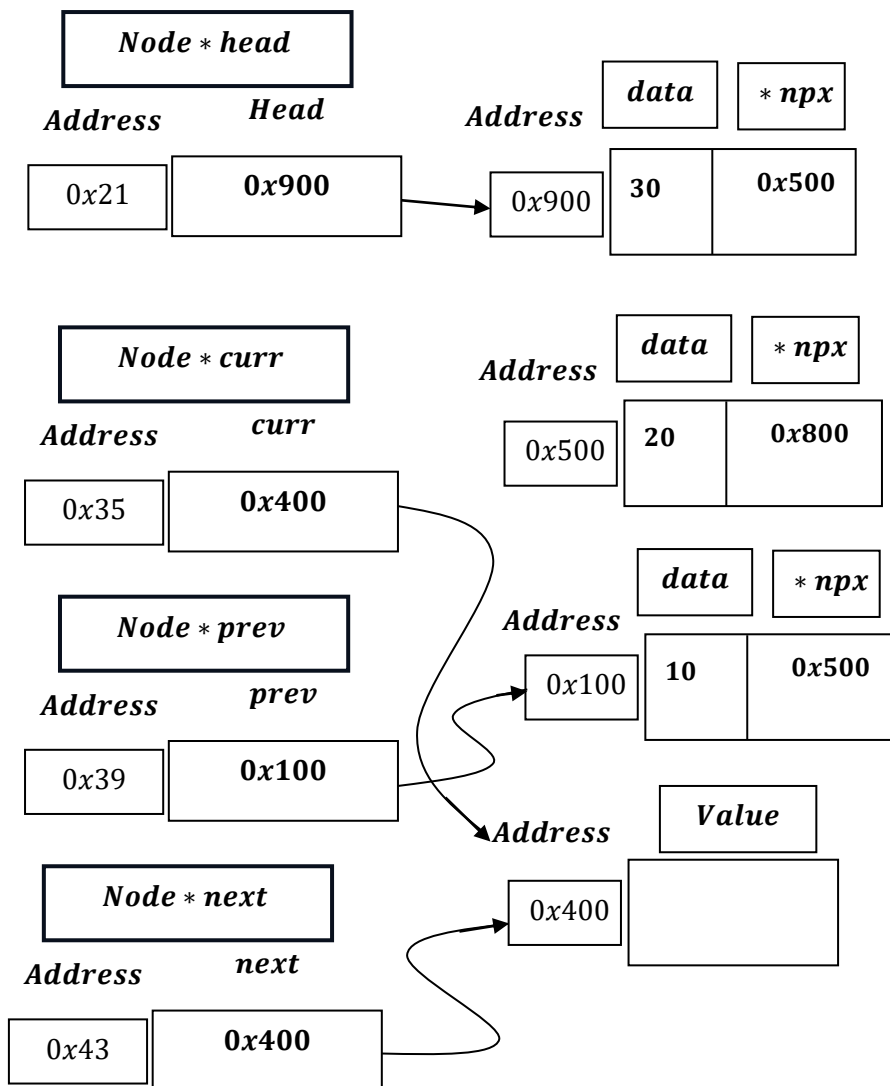
prev → npx = prevPrev: 0x500 ⊕ nullptr: 0x000

= 0101₂ ⊕ 0000₂ = 0101₂ = 0x500

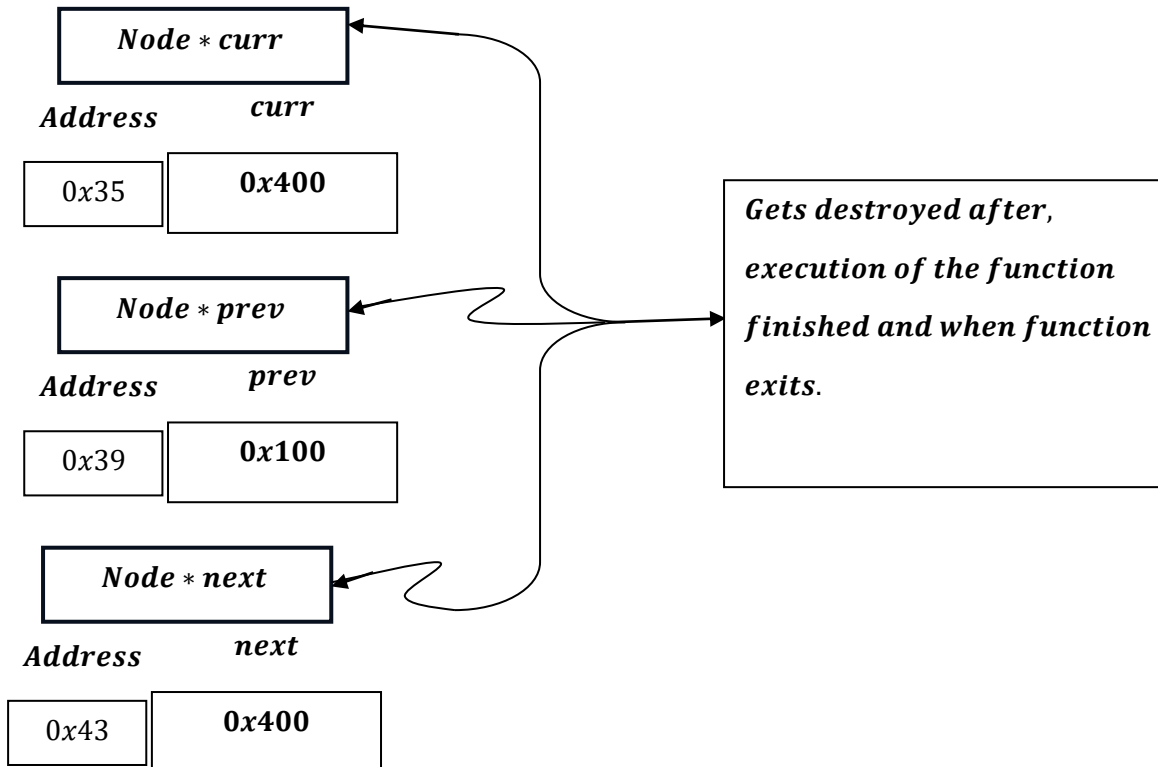


```
free(curr);
```

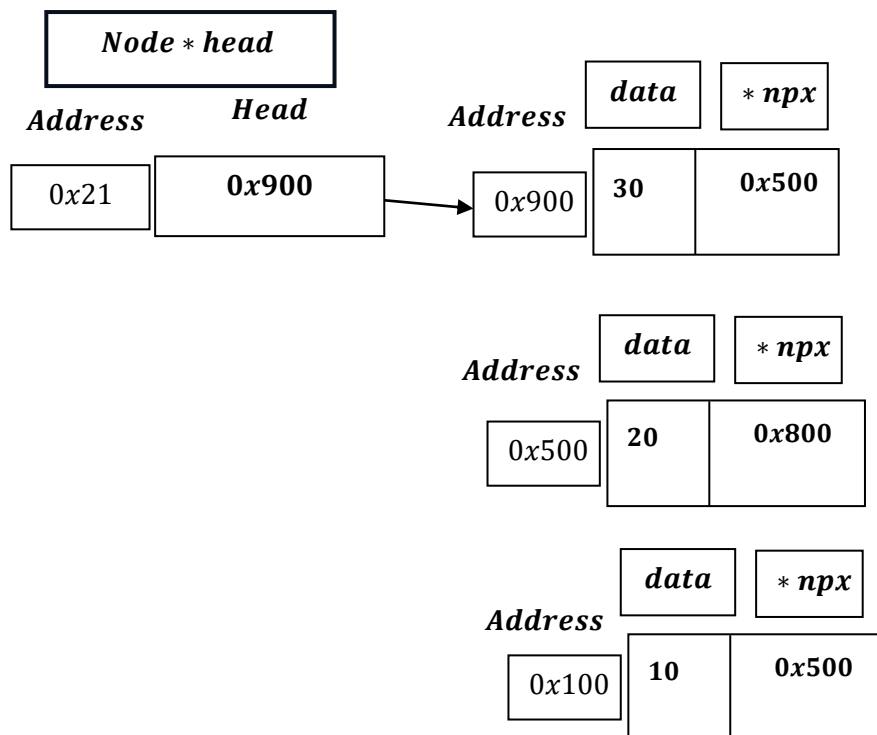
Free(curr: 0x400);



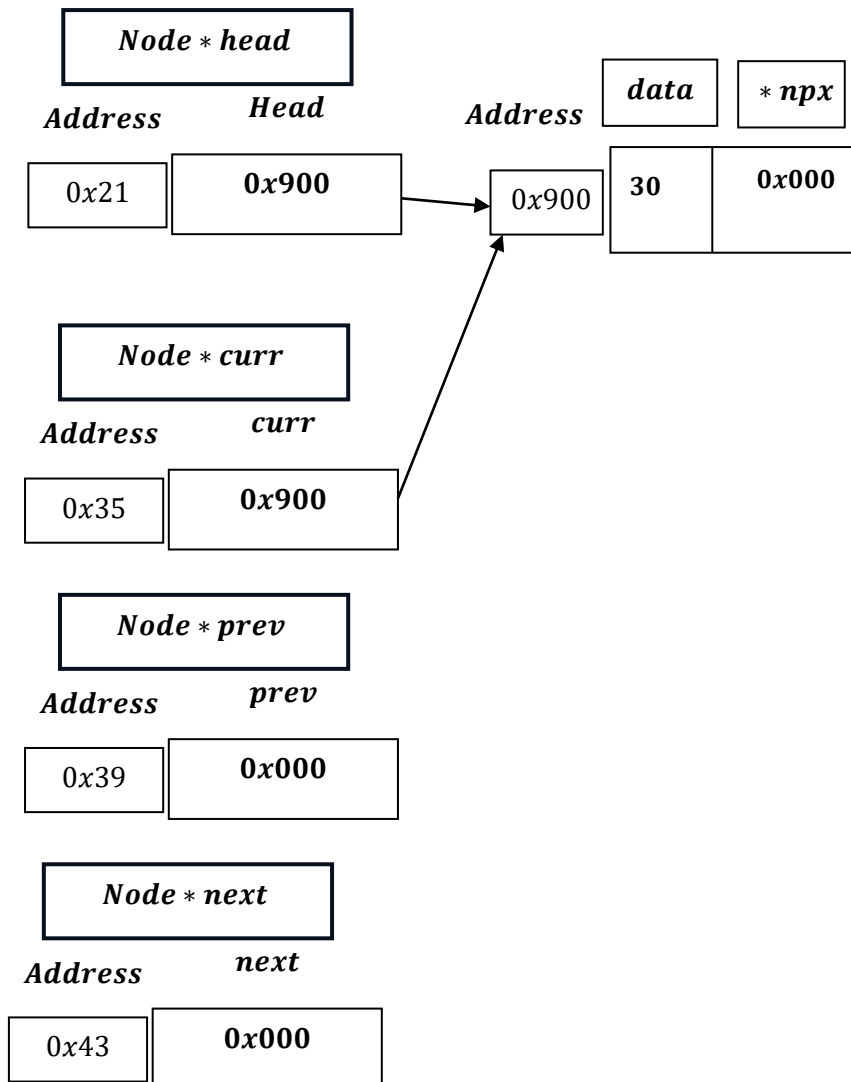
As , functions ends:



And we are left with:



Now say, when we have single node:



```
while (true)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr)
        break; // curr is the last node
    prev = curr;
    curr = next;
}
```

While(true):

next = XOR(prev: 0x000, curr → npx: 0x000)

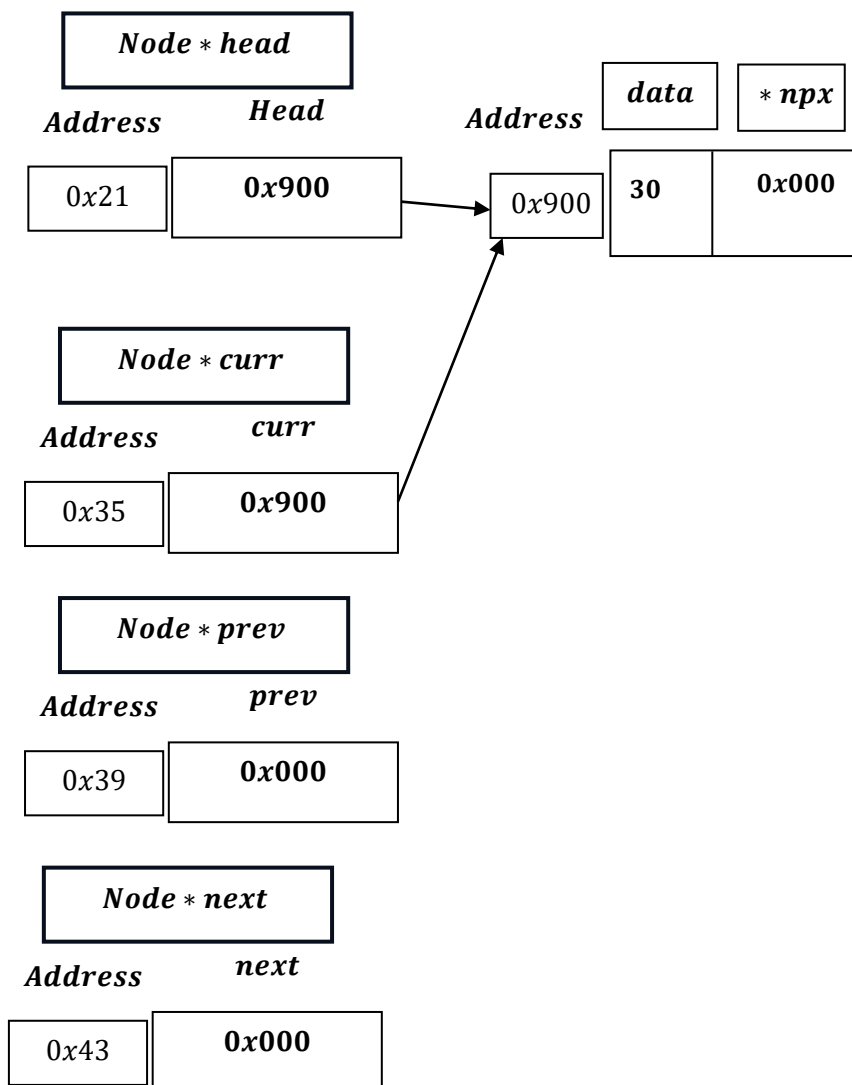
= 0000₂ ⊕ 0000₂ = 0000₂ = 0x000

if(next: 0x000 = nullptr: 0x000) is True:

break; → statement helps to exit from the loop.

... ..

... ..



```

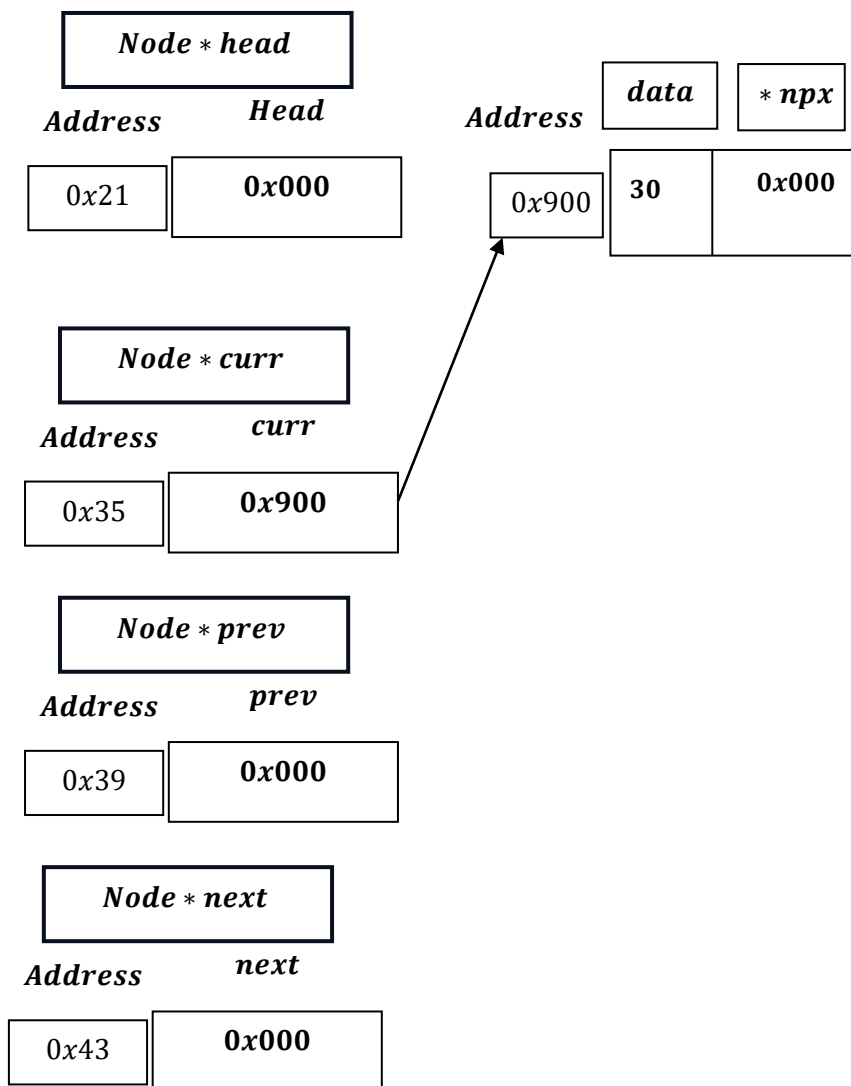
if (prev != nullptr)
{..... }
else
{
    head = nullptr;
}

```

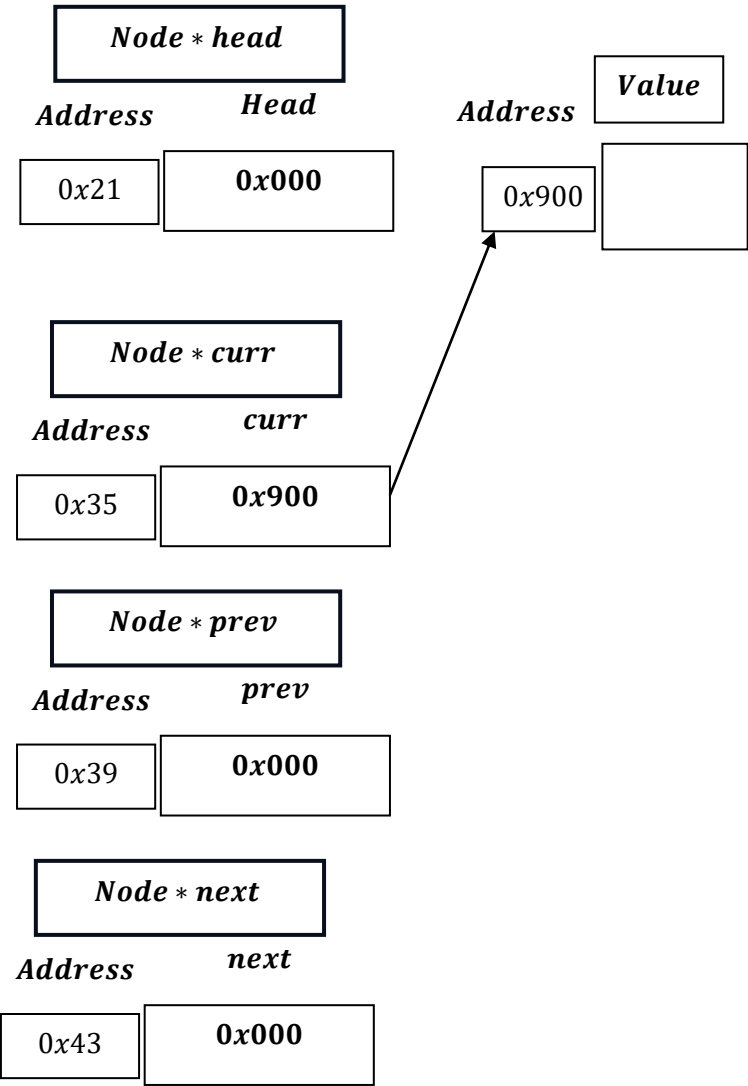
if (prev $\neq$ nullptr) false:

Else:

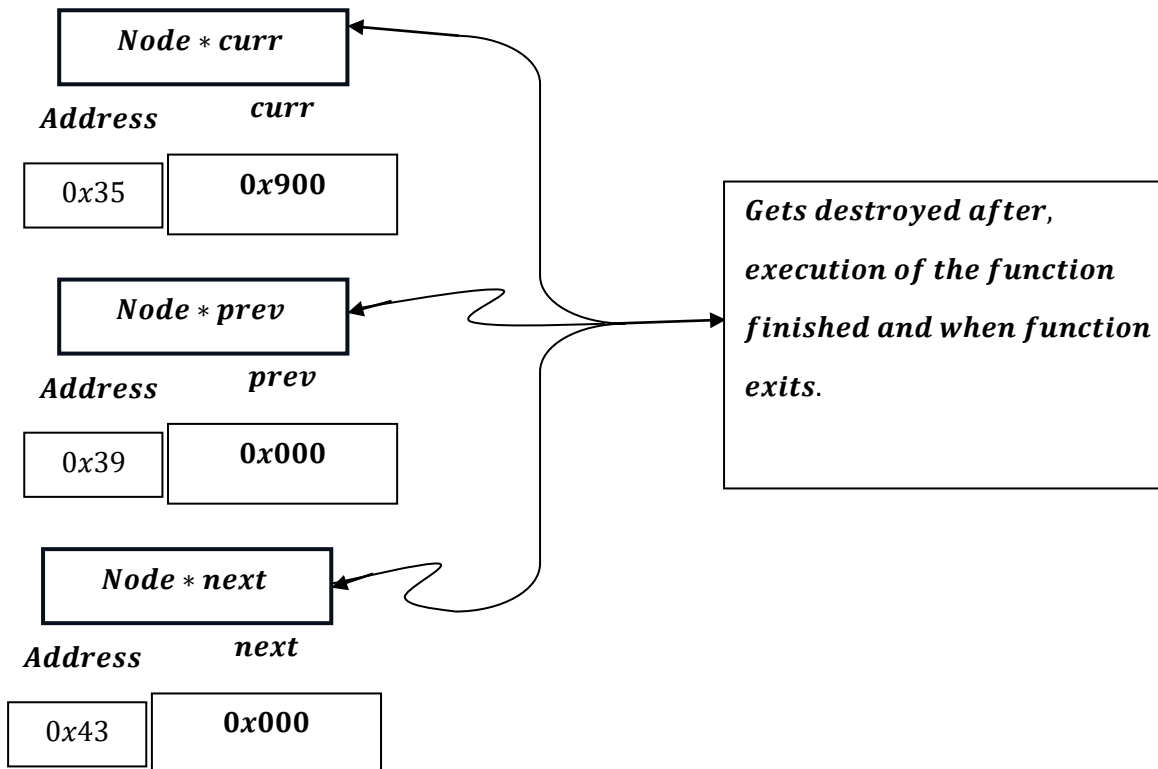
HEAD = NULLPTR = 0x000



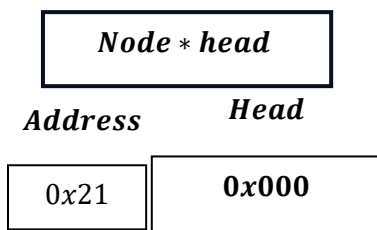
```
free(curr);  
Free(curr: 0x900);
```



As , functions ends:



And we are left with:



Time Complexity

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    while (true)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr)
            break;
        prev = curr;
        curr = next;
    }

    if (prev != nullptr)
    {
        Node *prevPrev = XOR(prev->npx, curr);
        prev->npx = XOR(prevPrev, nullptr);
    }
    else
    {
        head = nullptr;
    }

    cout << "Deleted " << curr->data << " from the end." << endl;
    free(curr);
}
```

The diagram illustrates the time complexity of the `deleteFromEnd()` function. It uses arrows to link specific code segments to complexity annotations:

- `if (head == nullptr)` is annotated with $O(1)$.
- The initialization of `Node *curr = head;`, `Node *prev = nullptr;`, and `Node *next;` is annotated with $O(1)$.
- The `while (true)` loop is annotated with *Runs `n` time.*
- The `if (next == nullptr)` condition within the loop is annotated with $O(1)$.
- The XOR operations `next = XOR(prev, curr->npx);` and `prev->npx = XOR(prevPrev, nullptr);` are annotated with $O(1)$.
- The final `cout` statement and `free(curr);` are annotated with $O(1)$.

Best Case Analysis[When List has no Node]

When list has no node, head pointed to `nullptr`, it is the best case $\Omega(1)$.

Worst Case Analysis[When List has numerous Node]

for loop:

while(true) $\rightarrow$ [this condition runs `n` times]:

next = XOR(prev, curr $\rightarrow$ npx) $\rightarrow$ runs $1 + 1 + 1 \dots (n - 1)$ times $= n - 1$.

if(next = nullptr): $\rightarrow$ checks `n` time. [including when false and true]

break; $\rightarrow$ runs `1` time.

prev = curr; $\rightarrow$ runs $1 + 1 + 1 + \dots (n - 1)$ times $= n - 1$

curr = next; $\rightarrow$ runs $1 + 1 + 1 + \dots (n - 1)$ times $= n - 1$

*$\therefore$ We focus innerstatement $= n - 1 + n + 1 + n - 1 + n - 1 = 3n - 3 + 2$
 $= 4n - 2$.*

$\therefore$ Time Complexity $= O(4n - 2) = 4 \times O(n) - O(2) = O(n)$.

Average Case

*We can say , if List is Empty ,
, we get our best case scenario, and when list is not empty and there present multiple nodes , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.*

In the case of DeleteAtEnd, the deletion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the deletion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence DeleteAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>DeleteAt End</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$
